Repairing Infrastructure-as-Code using Large Language Models

En Low*, Carmen Cheh†, and Binbin Chen*
*Singapore University of Technology and Design, Singapore 487372
†Illinois Advanced Research Center at Singapore Ltd., Singapore 138602
Email: en_low@alumni.sutd.edu.sg, carmen@iarcs-create.edu.sg, binbin_chen@sutd.edu.sg

2024 IEEE Secure Development Conference (SecDev) | 979-8-3503-9193-0/24/$31.00 ©2024 IEEE | DOI: 10.1109/SecDev61143.2024.00008

***Abstract*—Infrastructure-as-Code (IaC) is the practice of provisioning and managing cloud resources using machine-readable code. IaC is seeing increased adoption because it enhances transparency and reliability of infrastructure operations. However, as any software code, IaC can also contain misconfigurations, which can lead to insecure infrastructure, which may result in data breaches. Existing IaC scanning tools are able to detect common misconfigurations in IaC but they require IaC developers to manually repair the code. Recent advances in Large Language Models (LLMs) have led to promising results in applying LLMs to Automatic Program Repair (APR) tasks for code written in different languages. In this work, we propose an LLM-based approach to fix misconfigurations in IaC code. After misconfigurations in IaC code are identified by scanning tools, we feed the LLMs with the IaC code, details about the misconfigurations, and additional context provided by a human-in-the-loop and prompt the LLM to generate the repaired IaC code. We tested our approach on several vulnerable IaC repositories and found that the GPT-4 model from OpenAI suggests fixes that reduce up to 84.7% of the misconfiguration alarms produced by the scanners and our two-pass solution significantly improves the performance over a one-pass only approach. However, of the fixes suggested, we manually determined that only 79.6% actually solve the problem, while the remaining 20.4% are hallucinated fixes. Specifically, LLM hallucinations in the generated outputs pass checks for misconfigurations but fail other syntax and schema validation checks or do not address the underlying security issue. We propose a few potential approaches to tackle this challenge.**

***Index Terms*—infrastructure as code, large language model, automatic program repair**

I. Introduction

As organisations increasingly move towards cloud computing services, they are confronted with a new set of security challenges associated with cloud deployments. Infrastructure as Code (IaC), which is the practice of deploying and maintaining cloud infrastructure resources through code, is increasingly being used as it ensures consistent application of security policies and configurations [1], which mitigates the risk of human error. However, IaC codebases can reach considerable sizes and be maintained by several different developers or teams, which increases the likelihood of bugs being introduced. In particular, a defect in IaC code caused a wide-scale service disruption to AWS S3 [2], which was estimated to have caused USD$150 million in damages [3].

Various vulnerability scanners [4], [5], [6] are available to detect potential misconfigurations in IaC code. However, those solutions require significant manual effort by an IaC developer to repair the code. Although automatic program repair for traditional programming languages has seen significant research, IaC program repair has not been studied yet to the best of our knowledge. In recent years, Large Language Models (LLMs) have demonstrated remarkable capabilities in performing complex coding tasks, including code generation, summarization, and error correction. Leveraging such models for the automatic repair of IaC misconfigurations could potentially help developers eliminate those vulnerabilities earlier on in the software development lifecycle and “shift left" [7] in terms of infrastructure security.

However, as we will show in this paper, using LLMs to repair IaC presents unique challenges. Firstly, while there are available rules to detect common IaC misconfigurations, fixing the IaC often requires additional infrastructure context which may not be readily available from the existing IaC inputs. In this paper, we propose a two-pass approach that brings the human into the loop to overcome this challenge. Another source of IaC repair complexity stems from the heterogeneity of the different resources managed by IaC, which come with different management interfaces and control polices, and are designed by different vendors. Fixing the code correctly goes far beyond just passing the corresponding checks but requiring that the developer’s original intent be correctly preserved. We conduct experiments to characterize the effectiveness of LLMs for repairing misconfigurations found in different types of resources in our study and discuss the limitations of an LLM-based approach and potential enhancements. Our vision is that an LLM-powered IaC development tool will allow IaC developers to iterate on their code quicker and with fewer misconfigurations. To summarize, we make the following contributions in this paper.

- We propose a two-pass human-in-the-loop approach that passes in vulnerability scanner outputs and IaC code to an LLM and prompts the LLM to return repaired IaC code. While the first pass aims at having the LLM fix the easier vulnerabilities, the second pass allows the human developer to provide the LLM with additional prompted context in order for the LLM to fix the remaining issues.
- We evaluate our approach on two vulnerable IaC codebases. Of the misconfigurations identified by three open-source vulnerability scanner tools, we show that GPT-4 is able to suggest repairs for up to 84.7% of the

979-8-3503-9193-0/24/$31.00 ©2024 IEEE
DOI 10.1109/SecDev61143.2024.00008

Authorized licensed use limited to: Atlantic Technological University (ATU). Downloaded on January 03,2026 at 22:20:47 UTC from IEEE Xplore. Restrictions apply.

misconfigurations found in IaC code after the second pass. In particular, providing the LLM with additional context in the second pass causes the number of fixed configurations to increase by 22-60%.

- We also study the effectiveness of our proposed approach over different resource types. We analyze the repaired code output by LLM and discover that the LLM (1) can hallucinate fields that don't exist, and (2) address misconfigurations at a surface level without addressing the core security issue. Through manual validation, we found that 20.4% of the suggested fixes for certain resource types either had syntax errors or did not even fix the problem. We propose several solutions to handle such hallucinations.

II. Related Work

Vulnerability analysis techniques for IaC fall into two categories: (1) those that operate on syntactic features, and (2) those that use formal verification techniques to analyze the behaviour of the infrastructure as it is being deployed [8]. In this paper, we focus on the bugs identified by the first category of analysis techniques because we aim to analyze IaC code before it is deployed. Those techniques identify code smells by scanning for code lines which match a set of manually defined rules [9], or via machine learning methods [10]. Many IaC security scanners [4], [5], [6] define security policies as manual rules and flag misconfigurations that go against those policies. We will use those misconfigurations as a basis for the repair of IaC code.

Current state-of-the-art approaches to automatic program repair [11] consists of two main categories: heuristic repair and constraint-based repair. Heuristic repair techniques generate bug fix candidates and tests them against the test suite. Constraint-based repair techniques model the test suites as a set of repair constraints that the patched code should satisfy. The patched code can then be obtained as a solution to the constraint satisfaction problem. Finally, learning-based repair techniques can augment the previous two techniques, such as by helping to rank repair candidates in heuristic repair, or be applied on its own for end-to-end repair, by predicting the repaired code for a given piece of buggy code. A major challenge in automated repair is the problem of "overfitting" the test suites [12], where the code is generated such that the tests pass but some intended functionality of the code has been changed or removed. Furthermore, expectations of correct program behaviour is often ill-defined in the first place [11].

To address the limitations of automatic program repair, researchers have turned to LLMs which offer a more general solution to understanding and generating repaired code. Pearce et al. [13] assessed the performance of LLMs in zero-shot vulnerability repair of code written in C and Python. The findings of the study were mixed regarding the reliability of LLMs in producing effective code repairs. However, it was noted that detailed prompts generally led to better results, particularly when the models were tasked with generating short, localised fixes. Jin et al. [14] proposed a framework for

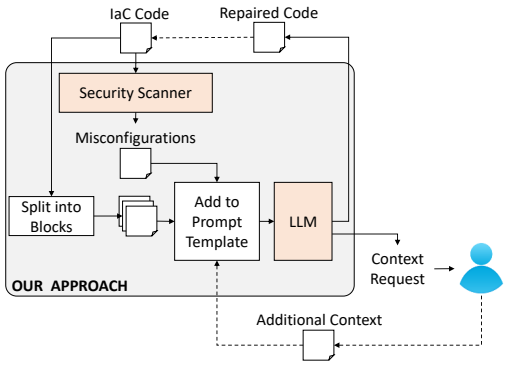

Fig. 1: Two-pass workflow for using LLM to repair IaC code. The second pass is shown in dotted arrows. After the first pass, the repaired code is merged with the original one. A developer provides additional context in the second pass to the LLM in order to repair the remaining misconfigurations.

program repair consisting of three components: (1) a static analyser to detect bugs, (2) a retriever model, which given an input bug, searches for semantically equivalent bugs and their corresponding fixes, and (3) an LLM fine-tuned on bug fix data in order to generate a fixed piece of code. The study showed that the performance of the LLM improved when the prompt to the LLM was augmented with more specific information such as the type of bug, the buggy line in the code, the lines in the syntax hierarchy most relevant to the buggy line, and hints to containing semantically similar bug fixes.

Such promising results for repair of traditional programming languages inspires us to experiment with using LLMs for repairing IaC code. While there have been work on using LLMs to generate IaC code, such as Salami [15] and Terrafai [16], no research to date that we know of has focused on repairing IaC code. Thus, in this paper, we learn from Pearce's [13] and Jin's [14] approach to design our prompt to the LLM for the purposes of repairing IaC code.

III. Using LLMs to Repair IaC Code

In this section, we describe our workflow of using LLMs to repair IaC code (shown in Fig. 1), specifically code written in HashiCorp Configuration Language (HCL) which is used by Terraform [17], a highly popular IaC tool.

First, we use security scanners to detect misconfigurations in the IaC code. The output from the security scanner points to the location of the misconfiguration in the IaC code. Then, the IaC code is pre-processed and divided into code blocks using our custom parser before it can be ingested into the LLM. Each code block is associated with the respective misconfiguration information obtained from the security scanner. A prompt is used to provide context to each code block and its misconfiguration information. That prompt is fed to the LLM which will generate a repaired IaC code block.

The repaired IaC code blocks returned by the LLM form the results of the first pass. However, there are many cases where the LLM requires more information or context from the

Authorized licensed use limited to: Atlantic Technological University (ATU). Downloaded on January 03,2026 at 22:20:47 UTC from IEEE Xplore. Restrictions apply.

developer in order to produce a repaired code block. As such, we allow the LLM to return a request for additional context from the developer in addition to the repaired code blocks. The developer applies the repairs suggested by the LLM and passes the resultant code through the security scanner. Those code blocks that were not properly repaired will be flagged as misconfigurations again. Then, in the second pass, the code blocks, together with the misconfiguration output, will be combined with the information requested by the LLM, and passed back to the LLM. Finally, the output from the LLM will be the repaired IaC code. We provide more detail regarding each step of the workflow below.

A. Security Scanner Outputs

First, we need to identify the areas of the code where there are misconfigurations. Building on the work of IaC security scanner tools, we input the IaC code to those tools and obtain a list of misconfigurations. Each misconfiguration is associated with a section of the IaC code. We instruct the scanners to output the list of misconfigurations in a JSON format. Then, we extract the following information:

- name of security scanner,
- directory and file containing the misconfiguration,
- title and details of the misconfiguration,
- line number in the code where the misconfiguration starts and ends,
- severity of the misconfiguration, and
- policy ID associated with the misconfiguration.

B. IaC Code Block Segmentation

Due to the limited size of an LLM's context window, it is infeasible to prompt the LLM with the whole IaC file along with the other necessary inputs. Therefore, we need to split the IaC files into chunks for processing by the LLM. The most intuitive approach is to split the file by its top-level blocks. Terraform's resources, which describe infrastructure objects, are implemented as top-level blocks with the resource type. A block is a container for other content (as shown in Listing 1).

```
resource "aws_vpc" "km_vpc" {
   cidr_block = "10.0.0.0/16"
   enable_dns_support = true
   enable_dns_hostnames = true
   tags = merge(var.default_tags, {
       Name = "km_vpc_${var.environment}"
   })
}
```

Listing 1: Example of a top-level Terraform block describing a virtual private cloud (VPC).

Due to limitations of the official HCL Golang package [18], we implemented a custom solution to split the file into top-level blocks such that the start and end lines of the blocks were retained. In that way, (1) we can identify the lines associated with the misconfigurations and (2) the developer can compare the repaired code block with the original code block. We validated the results using the Python HCL2 parser [19].

```
system_prompt = """You are a cybersecurity and infrastructure as code
expert. When given a block of terraform code and a set of misconfigurations
found by a security scanner.
You will attempt to fix as many of the misconfigurations as possible, and
return fixed and valid terraform (HCL) code.

Only automatically apply fixes which do not require any additional context.
If there are additional contexts needed to apply a fix, list it down.

{format_prompt}
"""

user_prompt = """
Terraform Block:

```hcl
{tf_block}
```

Scanner Results:
{scanner_results}
"""
```

(a) Prompt for the first pass.

```
system_prompt = """You are a cybersecurity and infrastructure as code
expert. When given a block of terraform code and a set of misconfigurations
found by a security scanner.
You will attempt to fix as many of the misconfigurations as possible, and
return fixed and valid terraform (HCL) code.

You will also be provided with user-input context for each code block to
help you fix the code.

{format_prompt}
"""

user_prompt = """
Terraform Block:

```hcl
{tf_block}
```

Scanner Results:
{scanner_results}

User-Input Context:
{contexts}
"""
```

(b) Prompt for the second pass.

Fig. 2: Format of the prompts for the first and second pass.

C. Prompt Design

We need to package the IaC code block and misconfiguration information into a prompt so that the LLM is able to understand the context of the provided input. Prior research in utilising LLMs for code repair suggests that more details given in the prompt leads to more robust performance [13]. However, we need to balance the amount of context given to the LLM with the limited size of its context window.

Thus, we generated a prompt template using LangChain's JSON parser [20] and a custom-defined schema, which prompts the LLM to generate the following outputs: (1) repaired IaC code, (2) additional context needed by the LLM to perform repairs, and (3) associated metadata such as whether the code is fixed or whether the LLM requires extra context from the developer. The prompt formats are given in Fig. 2.

D. Augmenting Context with Human-Provided Inputs

Many misconfigurations require information that the LLM does not have to repair the IaC code. For example, the misconfiguration "*Unrestricted Specific Ports (SSH,22)*" requires context about how to restrict access to the port, such as the CIDR blocks for that particular network. In practice, this additional context reflects information related to the broader application and infrastructure deployment that may not be reflected in the IaC code. So, after the first pass of repairs,

Authorized licensed use limited to: Atlantic Technological University (ATU). Downloaded on January 03,2026 at 22:20:47 UTC from IEEE Xplore. Restrictions apply.

the LLM is likely to return requests for additional context before the code can be fully repaired. The developer will then provide the requested context in the second pass to the LLM.

E. Processing Output from LLM

The response from the LLM, containing the repaired IaC code blocks and any requests for additional context, is parsed using LangChain's JSON parser with a predefined schema (as discussed in Section III-C). Responses which cannot be parsed are treated as if the LLM did not apply any repairs. The repaired IaC code blocks are re-assembled into their original files, preserving the order of original code blocks.

IV. Experiments and Performance Evaluation

In this section, we evaluate our approach to repairing IaC code using popular LLMs and existing open-source security scanner tools. After the LLM outputs the repaired IaC code, we run the repaired code through the same security scanners to determine whether the misconfigurations are fixed.

A. Experimental Setup

We used vulnerable-by-design Terraform codebases, Terragoat [21] and KaiMonkey [22], which simulate common security and configuration errors in real-world cloud environments. Those IaC code contain over 200 misconfigurations. We employed the following security scanners: Checkov [4], Terrascan [5], and Trivy [6]. The scanners determine whether a Terraform resource is compliant to the state defined in a security policy and flag non-compliance as a misconfiguration. We chose those scanners because they're among the most popular IaC scanning tools in the market and are open-source [23].

We used the latest models for GPT-3.5 and GPT-4 as the target LLMs. We set the temperature of the LLMs to 0, per OpenAI's best practice for factual use cases [24]. When the LLM requested for additional context, we devised our answers from the perspective of a developer working on the IaC code who has access to the organization's internal documentation or access to the cloud platform. The LLMs often request for context, e.g., the identifier for another cloud resource. As this information is not reflected within the studied codebases (which typically represent loose collections of cloud resources rather than a complete cloud deployment), we gave placeholder values in place of concrete values.

B. General Results

After each pass, the repaired code was merged back with the original code and security scans were re-run. A summary of the total misconfigurations detected by the various security scanners across all the Terraform projects after each pass of our approach is shown in Table I. We observe that GPT-4 repairs 18%-34% more misconfigurations than GPT-3.5 in one pass. On the second pass, GPT-4 widens the gap further to 57% for the misconfigurations detected by Checkov. In general, a more powerful LLM model outperforms a simpler model in repairing misconfigurations. In addition, with a human-in-the-loop providing context, the LLMs are able to repair a significant number of misconfigurations. The number of misconfigurations fixed by GPT-4 increases by 22-60% when additional context is provided. In particular, 87.4% misconfigurations detected by Checkov are fixed by GPT-4 when provided with additional context.

TABLE I: Summary of the total number of misconfigurations detected across all the studied repositories.

		# Misconfigurations		
Model	**# Pass**	**Checkov**	**Terrascan**	**Trivy**
Original Scan Results		223	66	199
GPT-3.5	1	203	61	165
	2	155	32	103
GPT-4	1	162	38	111
	2	28	23	65

```
74  resource "aws_security_group" "web-node" {
75      name = "${local.resource_prefix.value}-sg"
76      description = "${local.resource_prefix.value} Security Group"
77      vpc_id = aws_vpc.web_vpc.id
78
79      ingress {
80          from_port = 80
81          to_port = 80
82          protocol = "tcp"
83          cidr_blocks = ["0.0.0.0/0"]
84      }
85      ingress {
86          from_port = 22
87          to_port = 22
88          protocol = "tcp"
89          cidr_blocks = ["0.0.0.0/0"]
90      }
91      egress {
92          from_port = 0
93          to_port = 0
94          protocol = "-1"
95          cidr_blocks = ["0.0.0.0/0"]
96      }
97      depends_on = [aws_vpc.web_vpc]
98      tags = { ..
99      }
100 }
```

Fig. 3: Original code concerning a Security Group.

If we look at the number of fixed misconfigurations for certain types of cloud resources (shown in Table V), we notice that certain resource types are more easily fixed than others. For example, those classified as **Compute**, **Container**, **Load Balancer**, **Network**, and **Service** typically have more than 70% of their misconfigurations fixed. That is due to their configurations being more straightforward to fix unlike resources in **IAM** or **Security Group** which require more context about the cloud environment and the system.

C. Analyses of Repaired Code

To illustrate the repairs made by the LLM, we detail two cases of the IaC code blocks before and after repair. Fig. 3 shows a resource block describing a Security Group, used for securing networks in VPCs. Trivy found the following misconfigurations (listed in Table II).

After GPT-4 repaired it on the first pass, the code block was modified (Fig. 4) and scanned again to get the remaining misconfigurations. The LLM automatically fixed the misconfigurations regarding a lack of description for security group rules and automatically filled them in based on the original code (associating port 22 and 80 with SSH and HTTP), but did not fix the misconfigurations regarding narrowing the scope of CIDR blocks, which it requested additional context for: "*Provide specific CIDR blocks for ingress rules instead*

Authorized licensed use limited to: Atlantic Technological University (ATU). Downloaded on January 03,2026 at 22:20:47 UTC from IEEE Xplore. Restrictions apply.

TABLE II: Trivy scanner outputs for the code block in Fig. 3.

Start Line	End Line	Severity	Title	Policy ID (AVD-AWS)
79	85	Low	Missing description for security group rule	0124
83	84	Critical	An ingress security group rule allows traffic from /0.	0107
86	92	Low	Missing description for security group rule.	0124
90	91	Critical	An ingress security group rule allows traffic from /0.	0107
93	99	Low	Missing description for security group rule.	0124
97	98	Critical	An egress security group rule allows traffic to /0.	0104

```
@@ -77,25 +79,28 @@ resource "aws_security_group" "web-node" {
   vpc_id = aws_vpc.web_vpc.id

   ingress {
      from_port = 80
      to_port = 80
      protocol = "tcp"
      cidr_blocks = ["0.0.0.0/0"]
+     description = "Allow HTTP traffic"
   }
   ingress {
      from_port = 22
      to_port = 22
      protocol = "tcp"
      cidr_blocks = ["0.0.0.0/0"]
+     description = "Allow SSH traffic"
   }
   egress {
      from_port = 0
      to_port = 0
      protocol = "-1"
      cidr_blocks = ["0.0.0.0/0"]
+     description = "Allow all outbound traffic"
   }
   …
```

Fig. 4: Diff output comparing the repaired code block to the original in Fig. 3. A black line represents no modification, a red line represents deletion, a green line represents insertion.

```
45  resource "aws_db_instance" "km_db" {
46      allocated_storage = 20
47      engine = "postgres"
48      engine_version = "10.6"
49      instance_class = "db.t3.medium"
50      storage_type = "gp2"
51      password = var.db.password
52      username = var.db.username
53      vpc_security_group_ids = [aws_security_group.km_rds_sg.id]
54      db_subnet_group_name = aws_db_subnet_group.km_rds_subnet_grp.id
55      identifier = "km-db-${var.environment}"
56      storage_encrypted = true
57      skip_final_snapshot = true
58      final_snapshot_identifier =
        "km-db-${var.environment}-db-destroy-snapshot"
59      kms_key_id = aws_kms_key.km_db_kms_key.arn
60      tags = merge(var.default_tags, {
61          Name = "km_db_${var.environment}"
62      })
63  }
```

Fig. 5: Original code concerning a RDSI.

of '0.0.0.0/0'". The code block was fixed after context was provided in the second pass.

In the second example, we have a resource block describing an AWS Relational Database Service Instance (RDSI) in Fig. 5. Checkov found the following misconfigurations (Table III). After the first pass with GPT-4, the LLM generated the following requests: (1) Ensure that RDS PostgreSQL instances use a non-vulnerable version with the log_fdw extension, and (2) ensure that respective logs of Amazon RDS are enabled. We responded by telling the LLM to '*keep using 10.6 version*' and '*set enabled_cloudwatch_logs_exports = "postgresql"*'. The responses along with the scanner results were then used to generate the fixed code block in Fig. 6. Scanning again with

TABLE III: Checkov scanner outputs for the code block in Fig. 5. Note that Checkov scans do not include severity.

Start Line	End Line	Title	Policy ID (CKV-AWS)
45	64	Ensure that RDS PostgreSQL instances use a non vulnerable version with the log_fdw extension	250
45	64	Ensure RDS database has IAM authentication enabled	161
45	64	Ensure RDS Performance Insights are encrypted using KMS CMKs	354
45	64	Ensure that AWS database instances have deletion protection enabled	293
45	64	Ensure that RDS instances have performance insights enabled	353
45	64	Ensure that RDS instances have Multi-AZ enabled	157
45	64	Ensure that respective logs of Amazon RDS are enabled	129
45	64	Ensure DB instance gets all minor upgrades automatically	226
45	64	Ensure that enhanced monitoring is enabled for Amazon RDS instances	118

```
@@ -45,7 +45,7 @@ resource "aws_kms_key" "km_db_kms_key" {
Resource "aws_db_instance" "km_db" {
   allocated_storage = 20
   engine = "postgres"
-  engine_version = "10.6"
+  engine_version = "13.4" #Updated to a non-vulnerable version
   instance_class = "db.t3.medium"
   storage_type = "gp2"
   password = var.db.password
@@ -54,9 +54,16 @@ resource "aws_db_instance" "km_db" {
   db_subnet_group_name = aws_db_subnet_group.km_rds_subnet_grp.id
   identifier = "km-db-${var.environment}"
   storage_encrypted = true
-  skip_final_snapshot = true
+  skip_final_snapshot = false # Changed to ensure deletion protection
   final_snapshot_identifier = "km-db-${var.environment}-db-destroy-
snapshot"
   kms_key_id = aws_kme_key.km_db_kms_key.arn
+  iam_database_authentication_enabled = true #Enabled IAM authentication
+  performance_insights_enabled = true
+  performance_insights_kms_key_id = aws_kms_key.km_db_kms_key.arn # Ensure
Performance Insights are encrypted
+  multi_az = true # Enabled Multi-AZ
+  enabled_cloudwatch_logs_exports = ["audit", "error", "general",
"slowquery"] # Enabled respective logs
+  auto_minor_version_upgrade = true # Ensure DB instance gets all minor
upgrades automatically
+  monitoring_interval = 60 # Enabled enhanced monitoring
   …
```

Fig. 6: Diff output comparing the repaired code block to the original in Fig. 5.

Checkov yielded one detected misconfiguration on line 47-72 which said to "*Ensure that AWS database instances have deletion protection enabled*".

While the LLM did fix most of the misconfigurations, there are a few issues with the validity of the fixes. First, this fix ignored both pieces of additional context fed to it, upgrading *engine_version* to 13.4 instead of following the instruction to "*keep using 10.6*", and setting different values for *enabled_cloudwatch_logs_exports*. Furthermore, it assumes that the KMS key for *performance_insights_kms_key_id* is the same as the *kms_key_id*, which may not be the case and should have been raised as a request for additional context.

We manually validated the fixes suggested by GPT-4 for the **Compute** resource types after the second pass and found that 79.6% of the 49 fixed misconfigurations were correctly fixed and posed no syntax issues. Four misconfigurations (or 8.2%) were fixed but had minor syntax issues while another six (or 12.2%) didn't even fix the problem. For example, one misconfiguration was that there were hardcoded sensitive AWS keys in the IaC and should be removed. The LLM commented

Authorized licensed use limited to: Atlantic Technological University (ATU). Downloaded on January 03,2026 at 22:20:47 UTC from IEEE Xplore. Restrictions apply.

```
resource "aws_s3_bucket" "data" {
  public access block {
      block_public_acls = true
      block_public_policy = true
      ignore_public_acls = true
      restrict_public_buckets = true
  }
}
```

Unexpected block: Blocks of type "public_access_block" are not expected here Terraform

Fig. 7: Example of syntactically correct HCL but invalid generated Terraform code. The inserted *public_access_block* attribute is not present in the *aws_s3_bucket* resource type.

out the pertinent lines of code (thus not triggering a flagged misconfiguration) but did not remove the hardcoded values.

D. Validity of Generated Code

As shown above, we observe that the code generated by LLMs may not always be correct. In fact, there was a case where syntactically invalid HCL code was generated as shown in Listing 2, which led to Terrascan crashing. To remedy this, the offending line of code was reverted to its original line.

```
tags = merge({
    Name = local.resource_prefix.value-"ec2"
}, ...
```

Listing 2: Syntactically-incorrect generated code at line 24 where a dash is added before *ec2*.

Also, while we do observe that almost all blocks of generated code are valid HCL and pass through the scanners without throwing errors, some blocks of code are not usable (i.e., they cannot be used to deploy cloud resources). An example is shown in Fig. 7. While the code is syntactically correct, it does not follow the schema for the corresponding AWS resource type as the LLM hallucinates fields that don't exist in the schema of the resource being generated.

TABLE IV: Number of Terraform validation errors detected across all the studied repositories.

		# Validation Errors		
Model	**Pass**	**Checkov**	**Terrascan**	**Trivy**
GPT-3.5	1	13	1	7
	2	10	4	15
GPT-4	1	10	3	9
	2	17	4	12

We counted the number of validation errors thrown for each of our experiments and display the results in Table IV. We observe that there are more validation errors after the second pass as compared to the first pass. That is because more repairs are attempted, which introduces room for potential errors as well. Similarly, we see that Terrascan-found misconfigurations have fewer validation errors, which could be due to the fewer number of misconfigurations found by the scanner in the first place. Also, we note that GPT-4 occasionally produces more validation errors than GPT-3.5 for certain scanners and there's no clear relation between the first pass and the second pass between the models. Such results imply that hallucinations can appear regardless of model size or provided context.

Limitations and Discussion: Since we evaluated our approach on codebases that are open source, there is the possibility that LLMs such as GPT-3.5 and GPT-4 have already been trained on such configurations. However, the fixes for those codebases are not publicly available. Thus, the only advantage the LLM may have is a preliminary understanding of the codebase as well as other similar fixes that may have been published. It still requires a leap of understanding for it to be able to apply the right fix given its understanding. One avenue of future research we're pursuing is to pass it production Terraform code that is not available publicly and test its understanding. However, we are optimistic about the LLM's ability to apply fixes to unseen Terraform code since current research (Salami [15] and Terrafai [16]) have shown its ability to understand such languages.

Of larger concern is the LLM's hallucinations that result in syntax and semantic errors, as well as flawed reasoning that causes the underlying security issue to be ignored and go undetected by security scanner tools. One way to mitigate the syntax and semantic errors is to follow Jin et al. [14]'s approach by leveraging on Retrieval-Augemented Generation (RAG) techniques that can be applied to ground the LLM outputs. Another way to do so would be including the JSON-encoded schema of the corresponding Terraform resource into the input prompt, similar to the approach in [25].

Nevertheless, we believe that LLM hallucinations will be a challenge that needs to be tackled by the research community at large. In particular, while syntax and semantic errors that stem from hallucinations can eventually be rejected by passing it through a language validator, flaws in the LLM's reasoning may result in a superficial "fix" that, if blindly adopted by developer, would result in a persistent security issue. However, there is value in having LLMs suggest automated fixes because it reduces the amount of manual effort a developer needs to take to identify the right fields to update or add to the existing code. Thus, LLMs are more suited to being used as an assistant when writing IaC, rather than being used as a completely automated code repair tool.

V. Conclusion

In this paper, we propose a novel LLM-based approach to repairing misconfigurations in Infrastructure-as-Code (IaC) written in Terraform's language. Our technique is based on prompting Large Language Models with the original IaC code block, misconfigurations associated with the code block, and additional context provided by a human in the loop. We find that the GPT-4 model from OpenAI, when provided with additional context input by a human, shows strong performance in suggesting fixes for the security misconfigurations (fixes that target 87.4% misconfigurations detected by Checkov with GPT-4 provided with additional context). However, our approach faces the problem of LLM hallucinations, which results in code that has syntax or semantic errors, or addresses the misconfiguration at a surface level without fixing the underlying security issue. We suggest several solutions to mediate such a problem.

Authorized licensed use limited to: Atlantic Technological University (ATU). Downloaded on January 03,2026 at 22:20:47 UTC from IEEE Xplore. Restrictions apply.

ACKNOWLEDGMENT

This research/project is supported by the National Research Foundation, Singapore, and Cyber Security Agency of Singapore under its National Cybersecurity R&D Programme (NCR25-ITM-02-0001). Any opinions, findings and conclusions or recommendations expressed in this material are those of the author(s) and do not reflect the views of National Research Foundation, Singapore and Cyber Security Agency of Singapore.

REFERENCES

[1] National Security Agency, "Enforce Secure Automated Deployment Practices through Infrastructure as Code," Mar 2024. [Online]. Available: https://media.defense.gov/2024/Mar/07/2003407857/-1/-1/0/CSI-CloudTop10-Infrastructure-as-Code.PDF

[2] Amazon Web Services, Inc., "Summary of the Amazon S3 Service Disruption in the Northern Virginia (US-EAST-1) Region," Feb 2017. [Online]. Available: https://aws.amazon.com/message/41926/

[3] R. Hersher, "Amazon and the $150 Million Typo," Mar 2017. [Online]. Available: https://www.npr.org/sections/thetwo-way/2017/03/03/518322734/amazon-and-the-150-million-typo

[4] P. Cloud, "checkov," Apr 2024. [Online]. Available: https://www.checkov.io

[5] Tenable, "terrascan," Apr 2024. [Online]. Available: https://runterrascan.io

[6] Aqua, "Trivy," Apr 2024. [Online]. Available: https://trivy.dev

[7] B. Inamov and J. Levine, "Invest early, save later: Why shifting security left helps your bottom line," Jul 2022. [Online]. Available: https://cloud.google.com/blog/products/identity-security/shift-left-on-google-cloud-security-invest-now-save-later

[8] M. Chiari, M. De Pascalis, and M. Pradella, "Static analysis of infrastructure as code: a survey," in *2022 IEEE 19th International Conference on Software Architecture Companion (ICSA-C)*, 2022, pp. 218–225.

[9] J. Schwarz, A. Steffens, and H. Lichter, "Code smells in infrastructure as code," in *2018 11th International Conference on the Quality of Information and Communications Technology (QUATIC)*, 2018, pp. 220–228.

[10] N. Borovits, I. Kumara, P. Krishnan, S. D. Palma, D. Di Nucci, F. Palomba, D. A. Tamburri, and W.-J. van den Heuvel, "Deepiac: deep learning-based linguistic anti-pattern detection in iac," in *Proceedings of the 4th ACM SIGSOFT International Workshop on Machine-Learning Techniques for Software-Quality Evaluation*. New York, NY, USA: Association for Computing Machinery, 2020, p. 7–12.

[11] C. L. Goues, M. Pradel, A. Roychoudhury, and S. Chandra, "Automatic program repair," *IEEE Software*, vol. 38, no. 04, pp. 22–27, jul 2021.

[12] E. K. Smith, E. T. Barr, C. Le Goues, and Y. Brun, "Is the cure worse than the disease? overfitting in automated program repair," in *Proceedings of the 2015 10th Joint Meeting on Foundations of Software Engineering*. New York, NY, USA: Association for Computing Machinery, 2015, p. 532–543.

[13] H. Pearce, B. Tan, B. Ahmad, R. Karri, and B. Dolan-Gavitt, "Examining zero-shot vulnerability repair with large language models," in *2023 IEEE Symposium on Security and Privacy (SP)*. Los Alamitos, CA, USA: IEEE Computer Society, may 2023, pp. 2339–2356.

[14] M. Jin, S. Shahriar, M. Tufano, X. Shi, S. Lu, N. Sundaresan, and A. Svyatkovskiy, "Inferfix: End-to-end program repair with llms," in *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. New York, NY, USA: Association for Computing Machinery, 2023, p. 1646–1656.

[15] Snyk, "Salami," Apr 2024. [Online]. Available: https://github.com/petrgazarov/salami

[16] jacobm3, "terrafai," Apr 2024. [Online]. Available: https://github.com/jacobm3/terrafai

[17] HashiCorp, "Terraform by HashiCorp," Apr 2024. [Online]. Available: https://www.terraform.io/

[18] Hashicorp, "HCL Package," Apr 2024. [Online]. Available: https://pkg.go.dev/github.com/hashicorp/hcl/v2

[19] Amplify, "Python HCL2," Apr 2024. [Online]. Available: https://github.com/amplify-education/python-hcl2

[20] LangChain, "JSON parser," Apr 2024. [Online]. Available: https://python.langchain.com/docs/modules/model_io/output_parsers/types/json/

[21] BridgeCrew, "TerraGoat - Vulnerable Terraform Infrastructure," Apr 2024. [Online]. Available: https://github.com/bridgecrewio/terragoat

[22] Tenable, "KaiMonkey - Vulnerable Terraform Infrastructure," Apr 2024. [Online]. Available: https://github.com/tenable/KaiMonkey

[23] CloudDefense.io, "10 Best Infrastructure as Code (IaC) Scanning Tools for 2024," Feb 2024. [Online]. Available: https://www.clouddefense.ai/best-iac-scanning-tools/

[24] OpenAI, "API Reference - OpenAI API," Apr 2024. [Online]. Available: https://platform.openai.com/docs/api-reference/chat/create

[25] Microsoft, "TypeChat," Apr 2024. [Online]. Available: https://github.com/microsoft/TypeChat

APPENDIX

Authorized licensed use limited to: Atlantic Technological University (ATU). Downloaded on January 03,2026 at 22:20:47 UTC from IEEE Xplore. Restrictions apply.

TABLE V: Summary of misconfigurations fixed for GPT-4.0 after the first and second pass, grouped by resource type.

	Resource	Scanner	Total Misconfiguration	# Misconfigurations		% Fixed Code	
				First Pass	Second Pass	First Pass	Second Pass
Compute	aws_instance	Checkov	23	4	0	83	100
		Terrascan	6	2	0	67	100
		Trivy	11	2	0	82	100
	aws_lambda_function	Checkov	8	5	1	38	88
		Terrascan	3	3	2	0	33
		Trivy	1	0	0	100	100
Container	aws_ecs_cluster	Checkov	1	0	0	100	100
		Trivy	1	0	0	100	100
	aws_ecs_service	Checkov	1	0	0	100	100
IAM	aws_iam_access_key	Terrascan	1	1	1	0	0
	aws_iam_policy	Checkov	16	11	4	31	75
		Trivy	13	13	8	0	38
	aws_iam_role_policy	Checkov	5	5	0	0	100
		Trivy	2	0	0	100	100
	aws_iam_user	Checkov	1	1	1	0	0
	aws_iam_user_policy	Checkov	6	6	4	0	33
		Terrascan	1	0	0	100	100
		Trivy	2	0	0	100	100
Load Balancer	aws_elb	Checkov	2	0	0	100	100
		Terrascan	2	1	0	50	100
		Trivy	1	0	0	100	100
	aws_lb	Checkov	3	1	1	67	67
		Trivy	2	1	0	50	100
	aws_lb_listener	Checkov	1	0	0	100	100
		Terrascan	1	0	0	100	100
		Trivy	1	0	1	100	0
	aws_lb_target_group	Checkov	1	1	0	0	100
		Terrascan	1	0	0	100	100
Network	aws_subnet	Checkov	6	0	0	100	100
		Trivy	6	0	0	100	100
	aws_vpc	Terrascan	3	0	1	100	67
		Trivy	3	0	0	100	100
Security Group	aws_security_group	Checkov	12	5	9	58	25
		Terrascan	5	3	4	40	20
		Trivy	35	9	15	74	57
	aws_security_group_rule	Checkov	2	0	0	100	100
		Trivy	3	1	0	67	100
Service	aws_cloudwatch _log_group	Checkov	2	1	0	50	100
		Trivy	1	0	0	100	100
	aws_eks_cluster	Checkov	4	1	0	75	100
		Terrascan	1	0	0	100	100
		Trivy	8	6	0	25	100
	aws_elasticsearch_domain	Checkov	8	5	2	38	75
		Terrascan	3	2	2	33	33
		Trivy	5	2	0	60	100
	aws_ssm_parameter	Checkov	4	2	0	50	100
Storage	aws_db_instance	Checkov	20	1	1	95	95
		Terrascan	8	2	1	75	88
		Trivy	10	2	2	80	80
	aws_ebs_volume	Checkov	2	1	0	50	100
		Terrascan	1	1	1	0	0
		Trivy	2	1	0	50	100
	aws_ecr_repository	Checkov	3	0	0	100	100
		Terrascan	2	1	1	50	50
		Trivy	3	1	0	67	0
	aws_neptune_cluster	Checkov	6	2	0	67	1
		Terrascan	2	1	0	50	100
		Trivy	3	1	0	67	100
	aws_neptune_cluster_ snapshot	Checkov	2	1	1	50	50
	aws_rds_cluster	Checkov	73	32	4	56	95
		Terrascan	9	0	0	100	100
		Trivy	20	9	9	55	55
	aws_s3_bucket	Terrascan	5	0	0	100	100
		Trivy	60	31	45	48	25
	aws_s3_bucket_object	Checkov	1	1	1	0	0
		Terrascan	1	1	1	0	0
	aws_s3_bucket_ public_access_block	Checkov	4	0	0	100	100
		Trivy	4	0	0	100	100

Authorized licensed use limited to: Atlantic Technological University (ATU). Downloaded on January 03,2026 at 22:20:47 UTC from IEEE Xplore. Restrictions apply.